

Guia do ambiente local

Versão de 27/09/2026, a validar. Como rodar o Acadflow no seu computador para desenvolver e testar, com SQLite e sem Docker.

Os passos foram executados pelo Tech Lead no Windows com Python 3.12. A validação consiste em outra pessoa seguir só este texto, do começo ao fim, e anotar onde travou.

1. O que você precisa

Item	Versão	Observação
Git	qualquer recente	
Python	3.12	A produção usa 3.11. O 3.12 funciona para desenvolver
Acesso ao fork		Peça ao Tech Lead para ser adicionado como colaborador de <code>gustavosouto/PPGEC_rec</code>

2. Clonar o fork

```
git clone https://github.com/gustavosouto/PPGEC_rec.git Acadflow
cd Acadflow
git remote add upstream https://github.com/ldhonorato/PPGEC_rec.git
git checkout develop
```

Confira: `git remote -v` mostra `origin` apontando para `gustavosouto/PPGEC_rec` e `upstream` para `ldhonorato/PPGEC_rec`. `git branch` mostra `* develop`.

O `upstream` é o repositório do professor e serve só para leitura. Nunca faça push para ele.

3. Criar o ambiente virtual

O ambiente virtual isola as dependências do projeto do resto do seu computador.

Windows (PowerShell):

```
py -3.12 -m venv .venv
.venv\Scripts\Activate.ps1
```

Linux e macOS:

```
python3.12 -m venv .venv
source .venv/bin/activate
```

Confira: o terminal mostra `(.venv)` no começo da linha, e `python --version` mostra 3.12.

A pasta `.venv` já está no `.gitignore` do projeto e nunca entra num commit.

4. Instalar as dependências

```
python -m pip install --upgrade pip
pip install -r requirements.txt
```

Confira: `python -c "import django; print(django.get_version())"` mostra 5.2.

5. Criar o arquivo `.env`

Crie um arquivo chamado `.env` na raiz do projeto, ao lado do `manage.py`, com este conteúdo:

```
DEBUG=True
USE_POSTGRES=False
ACOMPANHAMENTO_ATIVO=True
```

- `DEBUG=True` liga o modo de desenvolvimento e desliga o redirecionamento para HTTPS.
- `USE_POSTGRES=False` faz o sistema usar SQLite, sem precisar instalar banco.
- `ACOMPANHAMENTO_ATIVO=True` liga o módulo da equipe no seu computador. Em produção ele fica desligado.

O `.env` já está no `.gitignore` do projeto. Nunca coloque nele senha de produção.

6. Preparar o banco local

O projeto guarda o banco SQLite no arquivo `db.sqlite3`, que está versionado no repositório do professor e tem dados de origem desconhecida. Não use esse arquivo. Os comandos abaixo mandam o Git ignorar mudanças nele no seu computador e criam um banco novo, só com as tabelas e os dados iniciais do sistema.

Windows (PowerShell):

```
git update-index --skip-worktree db.sqlite3
Remove-Item db.sqlite3
python manage.py migrate
```

Linux e macOS:

```
git update-index --skip-worktree db.sqlite3
rm db.sqlite3
python manage.py migrate
```

Confira: o `migrate` termina com `Applying sessions.0001_initial... OK`, e `git status` não mostra o `db.sqlite3` como alterado nem como apagado.

Nunca faça commit do `db.sqlite3`. Se um dia ele aparecer no `git status`, rode de novo o `git update-index --skip-worktree db.sqlite3`.

7. Criar o seu usuário

```
python manage.py createsuperuser
```

O sistema pede e-mail, nome e senha. O e-mail é o login. Use um e-mail que termine em `@ficticio.invalid`, por exemplo `seu.nome@ficticio.invalid`, para ficar claro que é um usuário de teste. Esse domínio nunca recebe e-mail.

8. Rodar os testes

```
python manage.py test
```

Confira: leva cerca de 2 minutos e termina com `OK`. Em 27/09/2026 eram 339 testes.

No Windows, o teste `ArquivoEnviadoTests` às vezes falha ao apagar uma pasta temporária no fim (`tearDownClass`). É uma falha conhecida do projeto, sem relação com o código da equipe. Rode de novo antes de investigar.

9. Subir o sistema

```
python manage.py runserver
```

Abra `http://127.0.0.1:8000/` e entre com o e-mail e a senha do passo 7. Para parar o servidor, use `Ctrl+C` no terminal.

Confira:

- a página de login mostra "Entrar | AcadFlow";
- depois de entrar, o menu lateral tem a seção "Coordenação";
- com `ACOMPANHAMENTO_ATIVO=True`, o menu tem o item "Acompanhamento", que abre a página do módulo. O item só existe depois que a fundação do módulo entrar na `develop`.

10. Limitações deste ambiente

- **Ações que mandam e-mail podem travar ou dar erro:** abrir processo, tramitar, dar ciência. O sistema entrega o envio ao Celery, que depende do Redis, e nenhum dos dois roda neste ambiente. Para navegar e desenvolver as telas da equipe, não é preciso.
- **O banco tem poucos dados:** 38 docentes, 2 alunos e 3 processos de exemplo. Os dados fictícios completos chegam com o comando que está sendo feito na Sprint 1.
- **Não é igual à produção:** produção usa Python 3.11, PostgreSQL 16, Unicorn, Celery e Redis. Antes de qualquer entrega ao professor, a equipe roda os testes em PostgreSQL 16 e o ensaio de produção.

11. No dia a dia

Antes de começar uma tarefa:

```
git checkout develop
git pull
git checkout -b feat/id-da-mudanca
```

As regras de branch, commit e pull request estão no `CONTRIBUTING.md`, na raiz do projeto.

Se o seu editor mostrar avisos como "Import django could not be resolved", aponte o interpretador Python do editor para o `.venv` do projeto.